

APPSEC-08: Workflows, Notifications and Remediation

Series: APPSEC — Application Security | **Notebook:** 8 of 10 | **Created:** June 2026 |
Last Updated: 09/29/2026

Overview

A security problem that sits in the Dynatrace UI helps nobody. The value of AppSec is realized when findings reach the people who can act on them — the SOC for triage, AppDev for code fixes, the platform team for infrastructure changes. **Workflows** are the bridge.

This notebook covers the trigger patterns, the routing decisions, the SLA + burn-rate alerting, and the ack-loopback that keeps Dynatrace and the ticketing system in sync.

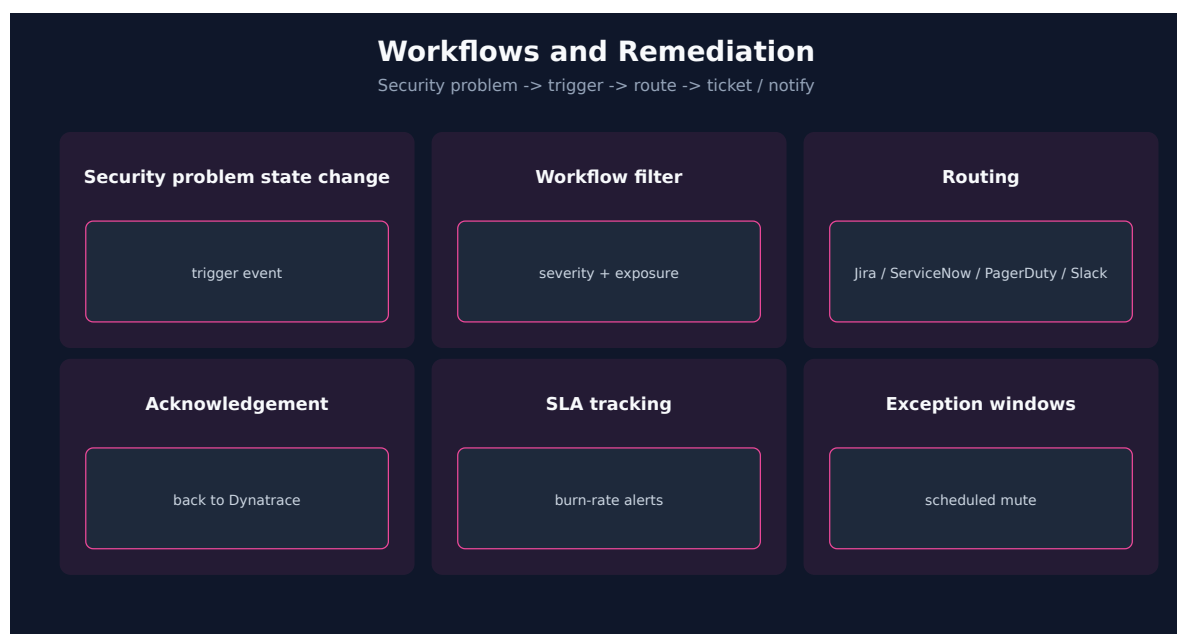


Table of Contents

1. [1. Triggers on Security Problems](#)
 2. [2. Routing: Jira / ServiceNow / PagerDuty / Slack](#)
 3. [3. Acknowledgement Loopback](#)
 4. [4. SLA and Burn-Rate Alerts](#)
 5. [5. DQL: Backlog Burn-Rate](#)
 6. [6. Next Steps](#)
 7. [References](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	Gen3 SaaS with Grail; AppSec entitlement enabled
OneAgent	Full-Stack mode (or code-module attached) on monitored hosts
Read access	At minimum <code>environment:roles:view-security-problems</code> and <code>storage:security.events:read</code> — see APPSEC-09 for the full model
Background	APPSEC-01 (fundamentals + three-pillar framing)

1. Triggers on Security Problems

Security routing uses the workflow **Event trigger** with event type `security.events` and a DQL matcher; a **Schedule** trigger covers digests and SLA sweeps.

Trigger	Filter (DQL matcher)	Use for
Event trigger on <code>security.events</code>	<code>event.type == "VULNERABILITY_STATUS_CHANGE_EVENT" and event.level == "VULNERABILITY"</code>	New / reopened / closed vulnerabilities — SOC triage

Trigger	Filter (DQL matcher)	Use for
Event trigger on <code>security.events</code>	<code>event.type == "DETECTION_FINDING" and product.name == "Runtime Application Protection"</code>	RAP attack alerting
Schedule	—	Digests and SLA sweeps

Do **not** use the **Problem trigger** for vulnerabilities. It fires on Davis problems — the correlated performance and availability problems — so a SOC workflow built on it never runs for a new vulnerability. Filter the vulnerability trigger to the vulnerability level (`event.level == "VULNERABILITY"`) so each vulnerability transition fires once, not once per affected entity; narrow further with `event.status_transition` (`NEW_OPEN`, `REOPEN`, `CLOSE`, `MUTE`, `UNMUTE` — an `experimental` field) and `vulnerability.risk.level` as needed. RAP detections are one record per observed attack, so that trigger needs a tight filter (production only, risk level) before it pages anyone.

Sources: [Event trigger \(DT docs\)](#) — “The Problem trigger starts a workflow when a problem opens, changes, or resolves.”, with Davis problems as its event source, and for `security.events`: “Use this type to automate remediation or ticketing when new vulnerabilities are detected.” (re-read 09/18/2026); [Vulnerability events \(DT semantic dictionary\)](#) for the status-change event and its transition values.

2. Routing: Jira / ServiceNow / PagerDuty / Slack

In community practice, a common routing split looks like this — verify it against your SOC's ownership model:

Destination	What goes there	Trigger filter
Jira	AppDev backlog: code-level vulns + third-party vulns scoped to service team	New/reopened vulnerability, risk level ≥ High, owner-tag mapped
ServiceNow	Platform/infra changes: SPM findings on cluster, cloud-account config	Severity ≥ High, framework in {CIS, PCI}

Destination	What goes there	Trigger filter
PagerDuty	Real-time SOC paging: RAP detections in production	<code>event.type == "DETECTION_FINDING" , product.name == "Runtime Application Protection" , production scope</code>
Slack	Awareness channel: new critical vulnerabilities, weekly digest	Risk level = Critical OR scheduled summary

A finding can fire into multiple destinations — there's no requirement to pick one. What matters is that each destination has a clear ownership boundary so no finding falls between teams.

Coming from classic security notifications. If your tenant still sends security alerts through the global notification settings in Settings (Classic), those do not carry over. The upgrade guide lists the difference in one line, *"You create and orchestrate alerts via Workflows"* versus *"You configure security notifications globally in Settings (Classic)"*, and gives the action as *"Recreate notifications using workflows."* The routing matrix above is where to start: a global classic notification usually becomes several narrower workflows, one per destination. The ready-made *Check your upgrade readiness* dashboard lists any legacy security notification that is still active.

Sources: [Application Security \(DT docs\)](#) for the workflow + notification framing, [Upgrade Third-party and Code-level vulnerabilities to the Latest Dynatrace \(DT docs\)](#) — the three quotes on classic security notifications.

3. Acknowledgement Loopback

When a Jira ticket is closed, the security problem in Dynatrace should reflect that. Without loopback, the Dynatrace open-problems count drifts from the ticketing system, and governance dashboards lose meaning.

Two implementation paths:

1. **Workflow-driven** — a workflow polls the ticketing system or subscribes to a webhook, then calls the Dynatrace API to acknowledge the security problem.
2. **API-driven from ticketing** — the ticketing system itself calls the Dynatrace API on close. Less common because it requires the security team to control ticketing-side

automation.

For programmatic ack, the IAM scope is `vulnerability-service:vulnerabilities:write` (on the Service User executing the workflow). See APPSEC-09 § 7 for the auth-scheme routing.

Sources: [IAM policy statements reference \(DT docs\)](#) for `vulnerability-service:vulnerabilities:write` (verified 06/04/2026). **Softened:** the two implementation paths are community practice; pick based on which team controls each side.

4. SLA and Burn-Rate Alerts

For regulated environments, security findings have remediation SLAs (e.g., Critical: 7 days, High: 30 days, Medium: 90 days). In community practice — borrowed from SLO burn-rate alerting — two complementary alerts are common; verify the pairing against your governance model:

1. **Per-problem SLA breach** — fires when an individual problem crosses its deadline.
2. **Backlog burn-rate** — fires when the *rate of new high-severity findings* exceeds the *rate of remediation* for several days running, signaling the team can't keep up.

Burn-rate alerts catch the systemic-overload pattern that per-problem alerts miss — a team can close every individual ticket on time and still be accumulating debt if new findings outpace remediation.

Sources: [Application Security \(DT docs\)](#) for the workflow + notification framing.

5. DQL: Backlog Burn-Rate

A starter query for backlog burn-rate dashboards: critical vulnerabilities opened (new or reopened) vs closed per day, over the trailing 7 days. It counts **status transitions** from `VULNERABILITY_STATUS_CHANGE_EVENT`, not state snapshots — RVA re-reports every open vulnerability at regular intervals, so counting `OPEN` snapshots would count the same vulnerability again on every report and inflate the "opened" side.

```
// Critical vulnerabilities: opened (new or reopened) vs closed per day, from
status transitions, last 7 days
fetch security.events, from:-7d
| filter event.provider == "Dynatrace"
| filter event.category == "VULNERABILITY_MANAGEMENT"
| filter event.type == "VULNERABILITY_STATUS_CHANGE_EVENT"
| filter event.level == "VULNERABILITY"
| filter vulnerability.risk.level == "CRITICAL"
| summarize opened = countIf(in(event.status_transition, {"NEW_OPEN",
"REOPEN"})), closed = countIf(event.status_transition == "CLOSE"), by:{day =
bin(timestamp, 1d)}
| sort day asc
```

Validation note: validated for syntax and field names on 09/18/2026 and executes cleanly; the validation tenant has no RVA data, so it returned 0 rows there. The `countIf` logic was additionally exercised on synthetic records.

Sources: [Vulnerability events \(DT semantic dictionary\)](#) — "A vulnerability change event is generated by Dynatrace Runtime Vulnerability Analytics (RVA) whenever a vulnerability's overall status or risk assessment changes", `event.status_transition` examples `NEW_OPEN` ; `REOPEN` ; `CLOSE` ; `MUTE` ; `UNMUTE` , and "RVA re-reports the open state at regular intervals and reports a resolved state once" (re-read 09/18/2026). **Dictionary:** `event.status_transition` (`experimental`), `vulnerability.risk.level` (`stable`), `event.type` (`stable`), read 09/18/2026. Because `event.status_transition` is `experimental` , re-check its values before building an alert on this query.

6. Next Steps

1. Decide which findings go to which destination. Write down the routing matrix before building the first workflow.
2. Build the ack-loopback before the first finding flows — drift starts on day one without it.
3. Read **APPSEC-09** to get the Service User permissions right for workflow execution.
4. Read **APPSEC-10** for the governance dashboards that read off the burn-rate query above.

References

Source	Coverage
Application Security (DT docs)	Workflow + notification framing
Upgrade Third-party and Code-level vulnerabilities to the Latest Dynatrace (DT docs)	Classic security notifications → workflows
Event trigger (DT docs)	Problem trigger vs Event trigger on <code>security.events</code>
Vulnerability events (DT semantic dictionary)	Status-change events and transition values
IAM policy statements reference (DT docs)	<code>vulnerability-service:vulnerabilities:write</code> scope

 **DISCLAIMER:** This information was AI generated and is provided "as-is" without warranty. It was produced as an independent, community-driven project and **not supported by Dynatrace**. Always refer to official [Dynatrace documentation](#) for the most current information.